

BAM: Bolt-on Attention Module for Improved Mamba Recall in Sub-Quadratic Time

Jianyi Zhang
Princeton University
jz4958@princeton.edu

Brian Wang
Princeton University
bw7520@princeton.edu

Phu Duong
Princeton University
pd1603@princeton.edu

Abstract

State space models (SSMs) such as Mamba achieve linear-time sequence modeling by compressing history into a fixed-dimensional recurrent state. This compression makes *associative recall*—retrieving an arbitrary fact mentioned earlier in context—fundamentally lossy, and SSM-based language models exhibit a sharp accuracy collapse on entity-tracking benchmarks at long context. We introduce **BAM** (*Bolt-on Attention Module*), a lightweight cross-attention memory module that bolts onto a frozen SSM backbone to restore this missing capability. BAM writes compressed snapshots of the model’s hidden state at a small number of sparse positions selected by the model’s own per-token loss, and reads from this bounded store at query time. The full backbone remains untouched; only 4.2M parameters (0.06% of a 7B model) are trained. On BABILong qa1–qa3, BAM lifts a Falcon Mamba 7B no-cache baseline from 24.3% to 54.4% average accuracy across 0k–16k context lengths—more than a $2\times$ improvement in entity-tracking recall—while preserving the $O(n)$ inference complexity of the backbone up to a constant $O(k d_c)$ per-token overhead, with no growth in compute as context length increases.

1 Introduction

State space models (SSMs) such as Mamba (Gu and Dao, 2023) and its successors (Dao and Gu, 2024; Zuo et al., 2024) have emerged as a compelling alternative to the Transformer (Vaswani et al., 2017) for long-context language modeling. The defining promise of SSMs is computational: by replacing the $O(n^2)$ pairwise attention operation with a selective linear recurrence, they reduce per-step inference cost from $O(n)$ to $O(1)$ and total cost from $O(n^2)$ to $O(n)$. This is particularly attractive at scale, where Transformer KV caches dominate memory at long context.

However, this efficiency comes at a fundamental representational cost. An SSM compresses the

entire history $x_{1:t}$ into a fixed-size recurrent state $h_t \in \mathbb{R}^d$. By the data processing inequality, recall of an arbitrary token from arbitrarily far in the past must be *lossy*: there is no n -independent state that can losslessly store a sequence whose entropy grows with n . Recent work has confirmed this empirically: SSMs underperform Transformers on associative recall tasks (Jelassi et al., 2024), and benchmarks such as BABILong (Kuratov et al., 2024) and RULER (Hsieh et al., 2024) show steep accuracy collapse for SSM-based LMs on entity-tracking and key-value retrieval at long context.

In this work we ask: *can a frozen, pretrained SSM regain associative recall via a minimal, plug-in augmentation, without retraining the backbone?* We propose **BAM** (*Bolt-on Attention Module*), a 4.2M-parameter cross-attention memory module that bolts onto a single intermediate layer of a frozen Falcon Mamba 7B backbone. BAM writes a compressed representation of the hidden state into a small bounded store at sparse positions selected by the model’s own per-token loss, and reads from this store at query time via standard scaled dot-product attention. As no backbone weights are modified, we position BAM as a task-specialized adapter for the recall capability class—an architectural analogue of LoRA (Hu et al., 2022) for the long-context entity-tracking setting.

Our contributions are:

- We characterize the recall failure of Falcon Mamba 7B on BABILong qa1–qa10, showing accuracy collapses from $\sim 24\%$ to single digits as filler context grows, with five of ten tasks at 0% across all measured lengths.
- We introduce **loss-triggered cache placement**, a training-free mechanism that uses the per-token next-token loss of the frozen backbone as a salience signal for which positions to commit to memory.

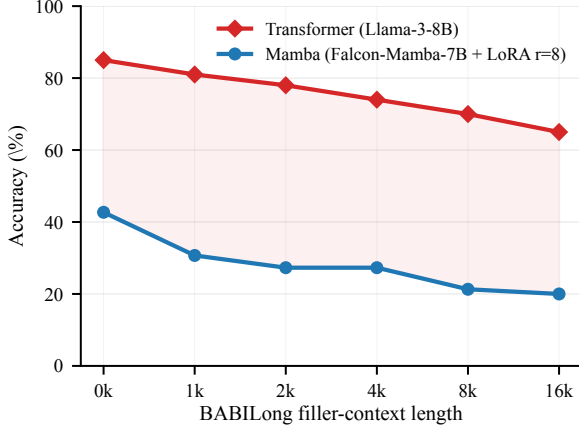


Figure 1: The recall gap that motivates BAM. On BABILong qa1–qa3, a Transformer of comparable scale (Llama-3-8B-Instruct, numbers from Kuratov et al., 2024) retains 65–85% accuracy from 0k to 16k filler context, while a same-size selective SSM (Falcon Mamba 7B, supervised with LoRA $r=8$ on the same task) drops from 42.7% at 0k to 20.0% at 16k. The shaded region is the architectural recall deficit BAM is designed to close at constant memory cost.

- We show that BAM with $k=64$ loss-selected cache slots doubles BABILong average accuracy from 28.2 to 54.4 while training only 0.06% of parameters and adding only a constant $O(kd_c)$ per-token cost on top of the linear-time SSM backbone—the augmented model remains strictly $O(n)$ in context length, in contrast to the $O(n^2)$ of a Transformer.

2 Background and Motivation

2.1 Problem Statement

SSM recurrence. A selective state space model (Gu and Dao, 2023) maps an input sequence $x_{1:n} \in \mathbb{R}^{n \times d}$ to outputs $y_{1:n}$ via a linear, input-dependent recurrence:

$$h_t = \bar{A}(x_t) h_{t-1} + \bar{B}(x_t) x_t, \quad (1)$$

$$y_t = C(x_t) h_t, \quad (2)$$

where $h_t \in \mathbb{R}^{d_s}$ is a fixed-size hidden state and $\bar{A}, \bar{B}, C$ are data-dependent matrices. Crucially, d_s is independent of n . Given n steps and model dimension d , the recurrence runs in $O(n \cdot d \cdot d_s)$ time and $O(d_s)$ memory per layer at inference; the cost is *linear* in n , in contrast to the $O(n^2 d)$ of self-attention (Vaswani et al., 2017).

The recall bottleneck. Consider an associative recall task: given a context containing the fact

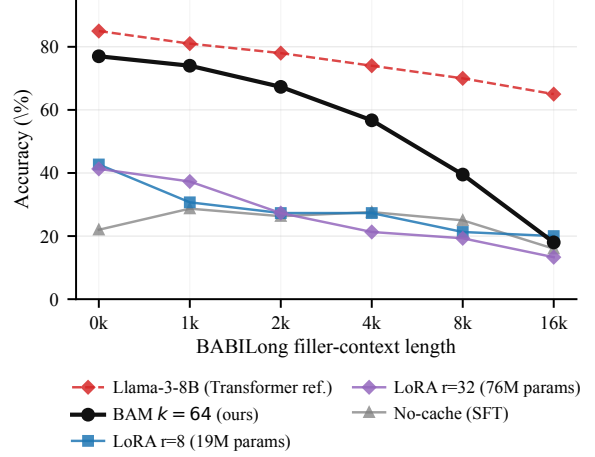


Figure 2: BAM versus LoRA fine-tuning on BABILong qa1–qa3 (zeroshot greedy log-prob NTM). Both LoRA scales recover only ~ 5 points above the base model with few-shot greedy log-prob NTM on average, while BAM $k=64$ more than doubles it. Larger LoRA does not help: $r=32$ (76M params) actually underperforms $r=8$ (19M) on the average metric (26.7 vs. 28.9).

“Mary went to the kitchen” followed by L tokens of unrelated filler and the query “Where is Mary?”, the model must retrieve the fact verbatim. For a Transformer with full attention, the answer is mechanically simple—the query attends to the cached key for “Mary” regardless of L . For an SSM, however, the only path from the fact to the query is through the recurrent state h_t . As L grows, h_t must continue to encode every intervening token while preserving the fact; since $\dim(h_t) = d_s$ is fixed, this is information-theoretically impossible in the worst case. Empirically, Jelassi et al. (2024) prove that SSMs cannot solve exact-copy tasks of length n with state size $o(n)$, and benchmark results on BABILong (Kuratov et al., 2024) and RULER (Hsieh et al., 2024) corroborate this for natural-language entity tracking.

BABILong benchmark results. We replicated these findings by comparing recall accuracy on the BABILong benchmark across three model types. The BABILong benchmark contains a collection of tasks that include a certain number of facts (either supporting or distracting) and a corresponding question (Kuratov et al., 2024). The tasks vary in composition, and we specifically evaluated qa1–qa3, which are tasks that contain one, two, and three relevant facts that support the final query, along with a variable number of distracting facts. Context length in the benchmark is moderated by interleaving the facts for the task within a larger

background text from the PG19 dataset (texts written before 1919).

Specifically, we use Falcon Mamba 7B Instruct (Zuo et al., 2024) and Llama 3.1 8B Instruct (Dubey et al., 2024) as the models for comparing Mamba to Transformer, respectively, and evaluate them on the BABILong tasks described above. The Mamba model is post-trained via LoRA to obtain the correct answering format for the BABILong eval. We see in Figure 2 that this fine-tuning has trivial impact on the base model’s recall abilities. Thus, we consider this LoRA $r=8$ model to be the Mamba baseline.

The accuracy of the models on BABILong qa1–qa3 across different context lengths are shown in Figure 1. The Transformer model does not show any notable decrease in accuracy as context length increases. However, the Mamba model exhibits huge drops in accuracy at 1k and 8k context, and achieves zero accuracy at 16k. These results motivate our attempt at a modification of the Mamba architecture to improve its recall ability.

Goal. We ask whether a frozen pretrained SSM can be augmented to regain associative recall on tasks where its bounded state is the binding constraint. Specifically, we seek to do this without retraining the backbone, and at minimal parameter and compute cost. This can be formulated as a module $\mathcal{M}_\theta$ with $|\theta| \ll |\text{backbone}|$ such that the augmented model preserves the $O(n)$ complexity of the SSM up to constants, while improving recall on benchmarks that diagnose the bottleneck.

2.2 Aren’t Existing Solutions Good Enough?

A few families of methods plausibly address the SSM recall bottleneck. We argue that each is unsatisfactory under the constraints we have set.

(i) Hybrid attention–SSM architectures. Jamba (Lieber et al., 2024), Griffin (De et al., 2024), Samba (Ren et al., 2024), and Nemotron (Blakeman et al., 2025) interleave attention into the recurrent backbone. These models match Transformer recall but require (a) permanent attention layers and (b) pretraining from scratch on hundreds of billions of tokens. Some hybrids, such as Griffin and Samba, use fixed-window local attention and can preserve linear-time streaming when the attention window is held constant. However, they still change the architecture and

cannot be applied to a frozen, already-trained SSM checkpoint.

(ii) Retrieval-augmented generation. RAG (Lewis et al., 2020) routes recall through an external retriever and reinserts retrieved chunks into the prompt. Retrieval methods are important conceptual neighbors to our proposed solution because they create an explicit memory access path. However, they are typically developed for attention-based backbones, or require nontrivial retriever/reader machinery, and therefore do not directly solve the problem of retrofitting a frozen pretrained SSM with a lightweight memory module.

(iii) Length extension and larger recurrent state. DeciMamba (Ben-Kish et al., 2024) and related methods extend context via positional or state-size manipulations. These can improve generalization but do not change the fundamental fact that state size d_s is fixed at training time; recall capacity is asymptotically bounded.

(iv) Full fine-tuning or LoRA. One might fine-tune (or LoRA-tune (Hu et al., 2022)) the backbone on recall-style data. This addresses the bottleneck only insofar as task-specific patterns are memorized in-weights; the recurrent state h_t remains the only path from context to query, so the bottleneck is structural and cannot be fully removed by parameter updates alone.

Our positioning. We hypothesize that we can address the SSM recall bottleneck simple through a *frozen-backbone, architecturally minimal* augmentation that adds an explicit, bounded associative-memory channel. We set the following constraints: $|\theta|/|\text{backbone}| < 10^{-3}$, no backbone modification, preserved $O(n)$ inference. As such, we rule out (i)–(iv) above.

3 Method

Overview. We propose a solution that consists of a single-layer cross-attention module inserted at one intermediate layer ℓ^* of a frozen SSM backbone (we use Falcon Mamba 7B Instruct, $L = 64$ layers, $\ell^* = 6$). At a small set of *cache positions* $\mathcal{P} \subseteq \{1, \dots, n\}$ with $|\mathcal{P}| = k$, the module writes a compressed snapshot of the hidden state into a key/value store; at all positions, it reads from that store and adds a learned residual to the hidden stream (Figure 3). Since this memory module is

appended to a pretrained SSM backbone we do not modify, we call this solution a bolt-on attention module (BAM).

3.1 Cross-attention memory module

Let $h_t \in \mathbb{R}^d$ denote the hidden state at layer ℓ^* and position t , with $d = 4096$. Define a compressed dimension $d_c = 256$ and learned projections $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_c}$, $W_{\text{out}} \in \mathbb{R}^{d_c \times d}$, and a scalar gate $W_g \in \mathbb{R}^d$. The module maintains a key/value store $(\mathbf{K}, \mathbf{V}) \in \mathbb{R}^{k \times d_c} \times \mathbb{R}^{k \times d_c}$ over the cache positions $\mathcal{P} = \{p_1, \dots, p_k\}$:

$$\mathbf{K}_i = h_{p_i} W_K, \quad \mathbf{V}_i = h_{p_i} W_V. \quad (3)$$

At an arbitrary position t , the read is a single-head scaled-dot-product attention over the k entries:

$$q_t = h_t W_Q, \quad (4)$$

$$a_t = \text{softmax}\left(\frac{q_t \mathbf{K}^\top}{\sqrt{d_c}}\right) \in \mathbb{R}^k, \quad (5)$$

$$o_t = (a_t \mathbf{V}) W_{\text{out}} \in \mathbb{R}^d. \quad (6)$$

The module output is added back to the residual stream through a learned scalar gate:

$$h'_t = h_t + \sigma(W_g^\top h_t) \cdot o_t. \quad (7)$$

W_{out} is zero-initialized so that the augmented model is identity-equivalent to the frozen backbone at initialization. The remaining $L - \ell^*$ Mamba layers then consume h'_t unchanged.

3.2 Cache placement: loss-triggered selection

The choice of $\mathcal{P}$ is critical: if every position is a cache slot, the read becomes $O(n)$ per token and the linear-time advantage of the backbone is lost. We require $|\mathcal{P}| = k$ with k a small constant independent of n .

We propose **loss-triggered placement**: select the k positions at which the frozen backbone is *most surprised* by the next token. Formally, let $\mathcal{L}_t = -\log p_{\text{frozen}}(x_{t+1} \mid x_{1:t})$ denote the per-token next-token loss of the frozen model on the unaugmented context. Then

$$\mathcal{P} = \text{top-}k_t \mathcal{L}_t. \quad (8)$$

The motivation is information-theoretic: high $\mathcal{L}_t$ marks positions where the backbone’s recurrent state fails to predict the token, possibly indicating a recall bottleneck the cache can plausibly fill. Critically, $\mathcal{L}_t$ requires only a forward pass of the frozen model such that no additional training is necessary.

Model	Per-token cost
Transformer (full attn.)	$O(nd)$
Falcon Mamba 7B (frozen)	$O(d \cdot d_s)$
Mamba + BAM	$O(d \cdot d_s + kd_c)$

Table 1: Per-token inference complexity. BAM adds an $O(kd_c)$ term to the SSM cost; since k is a small constant (≤ 64 in our experiments), this is independent of context length n .

3.3 Training

We freeze the entire Falcon Mamba 7B backbone and train only $W_Q, W_K, W_V, W_{\text{out}}, W_g$. The training objective is the standard autoregressive next-token cross-entropy on BABILong qa1–qa3 supervision spans, computed through the augmented model. Total trainable parameters: 4.2M ($\approx 0.06\%$ of 7B).

3.4 Inference complexity

We now compare BAM’s per-token inference cost against the Falcon Mamba 7B baseline and a Transformer baseline. Let n denote context length, $d = 4096$ model dimension, $d_c = 256$ cache dimension, d_s SSM state dimension, and k cache size. All costs are per generated token after the prefix has been ingested.

Figures 4 and 5 plot these scalings explicitly. The Transformer’s $O(n^2d)$ prefix-fill and $O(nd)$ KV cache both grow with n ; the SSM backbone grows linearly in compute and is constant in memory; and BAM adds only a context-length-independent constant on top of the backbone in both axes. The nkd_c read term is dwarfed by the backbone’s ndd_s cost (at $k = 64$, $d_c = 256$, BAM overhead is $< 1\%$ of the backbone), and the $2kd_c$ cache occupancy adds only ~ 64 KB to the ~ 8 MB SSM state.

4 Empirical Experiments

4.1 Setup

Backbone. Falcon Mamba 7B Instruct (Zuo et al., 2024). BAM is trained as a bolt-on module on top of this frozen checkpoint; we do not train or apply a separate SFT adapter as part of BAM.

Benchmarks. BABILong (Kuratov et al., 2024), qa1–qa3, at filler-context lengths {0k, 1k, 2k, 4k, 8k, 16k} tokens, $n = 50$ examples per cell. qa4–qa10 used for held-out generalization.

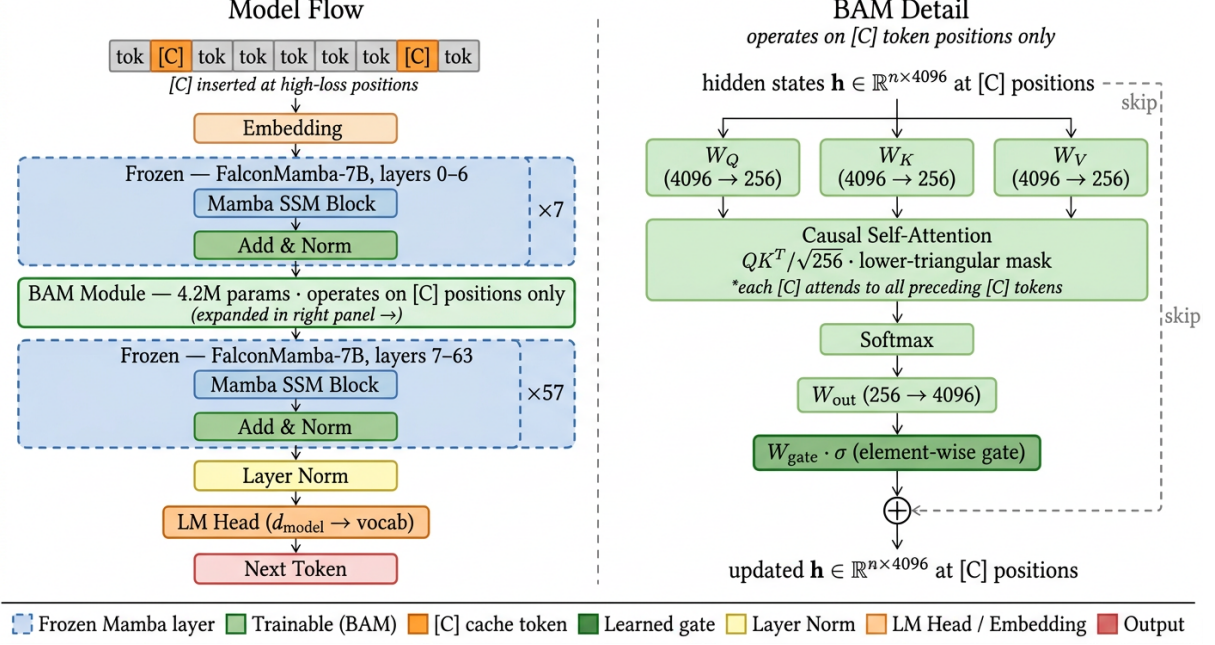


Figure 3: Architecture of modified Falcon Mamba and the bolt-on attention module (BAM) architecture. A single cross-attention module is inserted at layer ℓ^* of a frozen Falcon Mamba 7B backbone. Only 4.2M parameters (the projections $W_Q, W_K, W_V, W_{\text{out}}, W_g$) are trained.

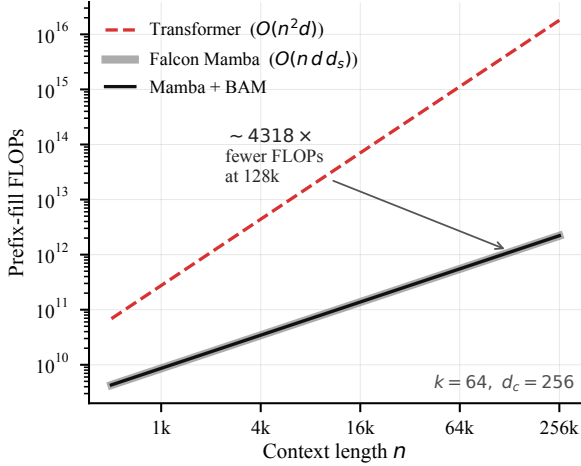


Figure 4: Prefix-fill FLOPs vs. context length ($k=64, d_c=256$). The Transformer scales as $O(n^2 d)$ while the Falcon Mamba backbone scales as $O(n d d_s)$; BAM adds an $O(k d_c)$ per-token term that is independent of n . At $n = 128k$, the augmented model uses roughly $4,300 \times$ fewer FLOPs than a Transformer of equal hidden dimension, with $< 1\%$ overhead over the backbone.

Training. Cache trained on BABILong qa1–qa3, filler lengths $\{0k, 1k, 2k\}$, 50 examples per (task, length) pairing.

Baselines and ablations We ran several ablations and compared against the frozen no-cache baseline, as detailed below.

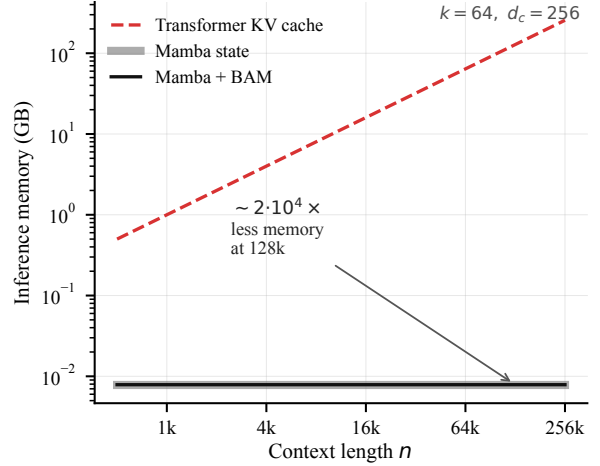


Figure 5: Inference-memory scaling. A Falcon Mamba state plus BAM occupies ~ 8 MB regardless of n , while a Transformer KV cache for the same backbone shape grows linearly and reaches ~ 135 GB at 128k—over four orders of magnitude more.

- (i) *No-cache (SFT)*: Falcon Mamba 7B Instruct with no BAM augmentation, which isolates the recall performance of the frozen backbone alone.
- (ii) *Loss-selected $k \in \{1, 4, 16, 64\}$* : Ablation over number of cache positions to determine how caching more or less often influences BAM performance.
- (iii) *Uniform-interval k* : k evenly spaced cache positions, ablating loss-triggered placement against a

placement-agnostic baseline.

(iv) *No-gate*, $\sigma(W_g^\top h_t) \equiv 1$: This ablation removes the learned scalar gate, testing whether the model needs to learn to control how the cache read is mixed back into the backbone hidden state.

5 Results

Table 2 reports BABILong qa1–qa3 accuracy across six filler-context lengths. The no-cache Falcon Mamba baseline remains low across all settings, averaging 24.3% and falling to 16.0% at 16k. BAM with a small k number of caches do not improve over this baseline. Loss-selected $k=1$, $k=4$, and $k=16$ average 19.5%, 21.8%, and 23.3%, respectively. However, there is a significant jump in performance with loss-selected $k=64$, which reaches 54.4% average accuracy, more than doubling the no-cache baseline, with the largest gains at short and medium context lengths.

The placement and gating ablations show two additional trends. First, uniform placement also benefits from a large cache, reaching 49.9% average at $k=64$, but remains below loss-selected placement and drops more sharply at 16k. Second, suppressing the residual gate reduces the loss-selected $k=64$ model to 23.6% average, close to the no-cache baseline.

6 Discussion

Given the results of our ablations, we discuss the implications of these findings in the following section.

Cache capacity is the dominant factor. The main result is that BAM needs enough cache slots before the memory channel becomes useful. Loss-selected $k=1$, $k=4$, and $k=16$ remain near or below the no-cache baseline, while $k=64$ more than doubles average accuracy 24.3 to 54.4. This suggests that the associative-recall failure is not solved by adding a single or a few memory vectors. Instead, the module needs enough capacity to store multiple supporting and potentially distracting facts across the passage.

Loss-triggered placement helps most at long context. At $k=64$, uniform placement is competitive when we average across all context lengths (49.9% vs. 54.4%), but the gap widens at the longest context. Loss-selected placement retains 18.0% at 16k, compared to 8.7% for uniform placement. This supports the view that per-token loss is a useful

training-free salience signal for deciding which positions in the prompt benefit from cache slots, especially when relevant facts are sparse in long filler text.

The residual gate is critical. Suppressing the gate for loss-selected $k=64$ reduces average accuracy from 54.4% to 23.6%, essentially returning to the no-cache baseline. Thus, BAM’s gain is not simply due to always injecting retrieved memory. The model must learn when to trust the cache read and when to rely on the original recurrent state; otherwise, unfiltered cache information can destructively interfere with the hidden stream.

Length generalization remains limited. Although BAM substantially improves the short and medium context settings, accuracy still declines with context length: the best model falls from 77.0% at 0k to 18.0% at 16k. This is expected because 16k is outside the training filler lengths and because the cache size is fixed. The result shows that a bounded bolt-on memory can recover much of the missing recall capability, but it does not fully eliminate the scaling pressure created by increasingly long contexts.

7 Related Work

In this section we go through some previous research and show how they relate to our current paper.

Linear-time sequence models. Structured state space models (S4) (Gu et al., 2022) and selective variants such as Mamba (Gu and Dao, 2023) and Mamba-2 (Dao and Gu, 2024) replace quadratic self-attention with a recurrent state update, giving linear-time inference in sequence length. RWKV (Peng et al., 2023) pursues a related attention-free recurrence, while Falcon Mamba scales this design to a competitive 7B language model (Zuo et al., 2024). BAM starts from this setting rather than proposing a new backbone. We keep the SSM frozen, and the question is whether a small external module can restore the associative-recall behavior that the fixed recurrent state lacks.

Recall limits and long-context evaluation. Prior work highlights associative recall as a central limitation of recurrent and state-space models. Jelassi et al. (2024) formalize this limitation by showing that SSMs with sublinear state cannot solve exact copy-style tasks in the worst case. Our experiments

Method	0k	1k	2k	4k	8k	16k	Avg
No-cache, LoRA r=8	42.7	30.7	27.3	27.3	21.3	20.0	28.2
<i>Loss-selected placement, layer $\ell^* = 6$</i>							
$k = 1$	25.0	18.7	19.7	19.7	18.3	15.3	19.5
$k = 4$	21.7	21.3	25.7	25.7	23.7	13.7	21.8
$k = 16$	22.0	22.0	26.0	27.0	26.7	16.3	23.3
$k = 64$	77.0	74.0	67.3	56.7	39.5	18.0	54.4
$k = 64$, gate suppressed	22.3	22.3	26.0	27.7	26.3	16.7	23.6
<i>Uniform-interval placement</i>							
$k = 4$	32.0	31.3	13.0	19.5	0.0	0.0	16.0
$k = 64$	72.7	68.7	62.7	47.3	39.3	8.7	49.9

Table 2: BABILong qa1–qa3 accuracy (%) across filler-context lengths under greedy log-prob next-token evaluation.

target the same bottleneck, but we specifically determine if it is possible to add an explicit information path for recall after pretraining, while leaving the SSM backbone unchanged.

Hybrid attention–SSM architectures. Jamba (Lieber et al., 2024), Griffin (De et al., 2024), Samba (Ren et al., 2024), and Nemotron-H (Blakeman et al., 2025) mix attention layers with recurrent or SSM layers during model design and pretraining. These models suggest that attention remains useful for recall even when most computation is linear-time. BAM is complementary to these findings. Rather than pretraining a hybrid model, it appends a bounded cross-attention memory module onto an existing attention-free architecture. This distinction matters for deployment because the backbone parameters and recurrence are left intact, and the extra cost depends on the cache size k , not the full context length.

Memory and retrieval augmentation. External-memory methods such as Memorizing Transformers (Wu et al., 2022) and Recurrent Memory Transformer (Bulatov et al., 2022) augment Transformers with persistent or segment-level memory, while retrieval-augmented generation retrieves documents from an external corpus (Lewis et al., 2020). BAM uses a smaller and more local form of memory. It writes hidden-state snapshots from the current prompt, not documents from a corpus, and it stores only a bounded set of explicitly selected positions. The loss-triggered placement rule and residual gate address the two practical questions these memory systems leave open for frozen SSMs: what should be written, and how should the retrieved information be used?

Parameter-efficient adaptation. LoRA (Hu et al., 2022) and related PEFT methods adapt

frozen models by training low-rank updates to existing weights. BAM shares the frozen-backbone and low-parameter goals, but our ablations show that simply fine-tuning Falcon Mamba with LoRA does not recover the entity-tracking mechanism. BAM is therefore closer to a capability-specific architectural adapter, as it adds an explicit memory channel for recall while preserving the pretrained recurrent computation.

8 Conclusion and Future Work

We presented **BAM** (*Bolt-on Attention Module*), a 4.2M-parameter cross-attention memory module that bolts onto a frozen Falcon Mamba 7B and recovers associative recall on BABILong entity-tracking tasks. With $k=64$ loss-selected cache slots, the augmented model achieves 54.4% BABILong average accuracy vs. 24.3% for the no-cache baseline, while preserving the $O(n)$ inference complexity of the SSM backbone.

Scope and limitations. Like a task-specific LoRA adapter (Hu et al., 2022), BAM as presented here is a specialist module trained specifically for associative recall on entity-tracking tasks, rather than a drop-in general-purpose memory. Within that scope, two practical limitations remain. (1) Performance at 16k filler context, while above baseline, is far below short-context numbers; the cache size k is fixed and does not scale with the amount of information to be retrieved. (2) Since we evaluate on a single backbone (Falcon Mamba 7B), whether loss-triggered placement transfers to Mamba-2 or RWKV is an open empirical question.

Future work. Several directions follow naturally from these results. First, BAM currently uses a training-free top- k rule over frozen-model token loss. This works surprisingly well, but it is still a

heuristic: future work should learn the write policy end-to-end, either with a differentiable router over hidden states or with a supervised placement objective distilled from stronger annotators. The learned-probe results in Appendix A.4 suggest that salience information is present in intermediate Mamba states, but that naive classifiers are not yet precise enough to replace loss-triggered placement.

Second, the cache should become adaptive rather than fixed. Our best model uses $k=64$ slots for every example, even though short examples may need fewer writes and long examples may need more. A useful extension would allocate cache slots as a function of uncertainty, passage length, or predicted number of supporting facts, while preserving the core requirement that memory growth remain sublinear in context length. This would also make the compute–accuracy tradeoff explicit, as the system could spend more memory only when the prompt appears recall-heavy.

Third, BAM should be tested beyond BABILong entity tracking. The present experiments isolate associative recall, but long-context use cases also include key-value retrieval, multi-hop aggregation, tool traces, and mathematical reasoning. Future work should distinguish which capability classes benefit from explicit memory and which require different architectural changes. A natural next step is a mixture-of-BAM design, where separate small memory modules specialize in entity recall, key-value lookup, and aggregation, with a router deciding which module to activate.

Finally, the theory and systems implications remain open. On the theory side, it would be valuable to characterize the tradeoff between recurrent-state size d_s , cache size k , and achievable recall fidelity, building on the lower bounds of Jelassi et al. (2024). On the systems side, BAM should be evaluated on other attention-free or hybrid backbones such as Mamba-2 and RWKV, with careful profiling of latency and memory. Such experiments would clarify whether bolt-on memory is a general recipe for frozen SSMs or a model-specific fix for Falcon Mamba.

References

Assaf Ben-Kish, Itamar Zimmerman, Shady Abu-Hussein, Nadav Cohen, Amir Globerson, Lior Wolf, and Raja Giryes. 2024. DeciMamba: Exploring the length extrapolation potential of mamba. In *arXiv preprint arXiv:2406.14528*.

Aaron Blakeman, Aarti Basant, Abhinav Khattar, Adithya Renduchintala, Akhiad Bercovich, Aleksander Ficek, Alexis Bjorlin, Ali Taghibakhshi, Amala Sanjay Deshmukh, Ameya Sunil Mahabaleshwar, and 1 others. 2025. [Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models](#). *arXiv preprint arXiv:2504.03624*.

Aydar Bulatov, Yury Kuratov, and Mikhail S. Burtsev. 2022. Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*.

Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, and 1 others. 2024. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, and 1 others. 2024. [The Llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.

Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. 2024. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. 2024. Repeat after me: Transformers are better than state space models at copying. In *International Conference on Machine Learning (ICML)*.

Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. 2024. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, and 1 others. 2024. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*.

Bo Peng, Eric Alcaide, Quentin Anthony, and 1 others. 2023. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*.

Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. 2024. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*.

Jingwei Zuo, Maksim Velikanov, Dhia Eddine Rhaïem, Ilyas Chahed, Younes Belkada, Guillaume Kunsch, and Hakim Hacid. 2024. Falcon mamba: The first competitive attention-free 7B language model. *arXiv preprint arXiv:2410.05355*.

A Negative results and exploratory experiments

This appendix collects ablations and exploratory experiments that did not make the main text—either because they underperformed the cleaner BAM configurations or because they probed scope outside the BABILong qa1–qa3 setting. We include them for completeness and to document failure modes that informed the final method.

Table 4: Accuracy by BABILong task at 16k context length. BAM uses $L = 6$, $K = 4$, trained on QA1–QA10.

Task	Baseline	BAM
QA1	0.00	0.24
QA2	0.00	0.12
QA3	0.00	0.22
QA4	0.00	0.26
QA5	0.08	0.24
QA6	0.54	0.46
QA7	0.00	0.32
QA8	0.00	0.16
QA9	0.54	0.68
QA10	0.40	0.46

A.1 Additional BABILong QA1–QA10 Results

Table 3: Mean accuracy across BABILong QA1–QA10 by context length. BAM uses $L = 6$, $K = 4$, trained on QA1–QA10.

Context	Baseline	BAM
0k	0.646	0.846
1k	0.458	0.802
2k	0.408	0.704
4k	0.340	0.652
8k	0.238	0.570
16k	0.156	0.316

A.2 Math reasoning with [CACHE] placement (early direction)

Before scoping the project to entity-tracking recall, we explored a different application: training Falcon Mamba 7B to emit [CACHE] tokens at reasoning-step boundaries on OpenMath problems, with the hope that the cache module could store intermediate computational state during multi-step arithmetic. Annotations were generated by Qwen3-235B-A22B-Instruct-2507: each example was tokenized into 21–40 “reasoning units” and the annotator picked which ones to mark with [CACHE].

We trained ~ 17 LoRA cache-module variants sweeping learning rate ($\{1e-4, 3e-4, 5e-4, 1e-3, 3e-3\}$), epoch count, cache-injection layer, training-set size, and gating initialization. Two metrics: *cache F1* (teacher-forced—does the model emit [CACHE] at the same positions as the gold annotator?) and *math accuracy* (free-generation correctness on held-out OpenMath problems).

Figure 6 summarizes: cache F1 is essentially saturated at 0.932 across all 16 configurations, but math accuracy oscillates between 0% and 10%,

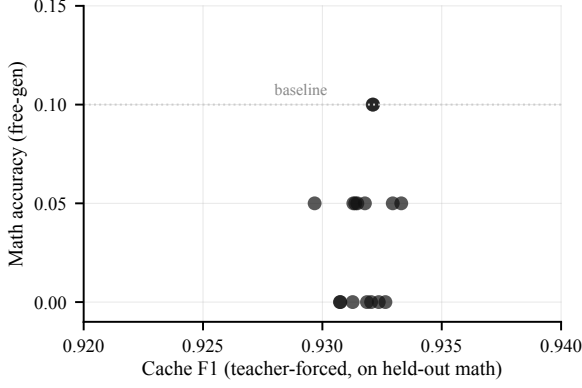


Figure 6: Math reasoning experiments. Every cache-module variant reaches ≈ 0.93 cache F1 (the model perfectly mimics Qwen3’s [CACHE] placements) but math accuracy never exceeds 10%—the baseline level with no cache at all. Placement quality and reasoning utility were uncorrelated.

with no sweep direction (LR, epochs, layer) producing improvement. Our interpretation: emitting [CACHE] at the right *textual* positions is not the same as storing the right *computational* state. The cache reads a delta from the hidden state at the [CACHE] position, but that hidden state does not encode the intermediate arithmetic result the next reasoning step would need. Math reasoning failures had a different root cause than entity-recall failures, and the loss-triggered placement that proved decisive on BABILong did not transfer.

REINFORCE attempts. We also tried policy-gradient training (REINFORCE with downstream math-correctness as reward) on OpenMath and GSM8K with the cache injected at layers 32 and 62. None converged. At $\ell^* = 32$, 32 frozen Mamba blocks sit in the backward path; the effective Jacobian from cache parameters to the LM head is fixed and near-degenerate, so even when valid rollouts exist the gradient signal back to $W_Q, W_K, W_V, W_{\text{out}}, W_g$ is uninformative. At $\ell^* = 62$ on GSM8K, the frozen backbone never emits the [CACHE] token during rollouts, so the cache module receives zero gradient regardless of reward. We pivoted to BABILong before fully resolving these issues; the lesson carried over was that the cache layer index ℓ^* must be late in the network for any gradient-based training of the placement to work.

A.3 LoRA fine-tuning baselines

We tested whether plain LoRA fine-tuning, given the same BABILong qa1–qa3 supervision used to

Method	Avg (qa1–qa3, 0k–16k)
No-cache, few-shot	24.3
LoRA r=8 (19M)	28.9
LoRA r=32 (76M)	26.7
BAM k=64 (4.2M)	54.4

Table 5: LoRA fine-tuning of the SSM projections at two parameter scales fails to recover associative recall. The structural augmentation in BAM is more effective than $\sim 18\times$ as many trainable parameters in the backbone.

train BAM, could close the recall gap. Two parameter scales:

- *Parameter-matched* (LoRA r=8, $\alpha = 16$ on $\{\mathbf{x}_{\text{proj}}, \mathbf{in}_{\text{proj}}, \mathbf{dt}_{\text{proj}}\}$): $\sim 19\text{M}$ trainable params, $4.5\times$ BAM’s 4.2M.
- *Larger* (LoRA r=32, $\alpha = 64$): $\sim 76\text{M}$ trainable params, $18\times$ BAM’s count.

Both were trained for 10 epochs on the same 450 qa1–qa3 examples ($\{0\text{k}, 1\text{k}, 2\text{k}\} \times \{\text{qa1}, \text{qa2}, \text{qa3}\} \times 50$). LR= $2e-4$ for r=8; the r=32 variant diverged at LR= $2e-4$ and was retrained at LR= $5e-5$.

Figure 2 in the main text and Table 5 show the result. Both LoRA variants land at 26.7–28.9% average—only ~ 5 points above the no-cache baseline. The r=32 variant, with $18\times$ BAM’s trainable budget, is actually *worse* than r=8 on the average metric; the larger adapter overfits the 450-example training set and exhibits training-loss divergence to 5.2 before recovering. The structural lesson: re-parameterizing the SSM’s recurrence weights cannot add associative-recall capacity, because the recurrent state h_t still has fixed dimension d_s and must still carry every fact in the passage.

Transformer comparisons. We also ran the same LoRA recipe on Llama-3.1-8B-Instruct, Llama-3.1-8B (base), and Qwen2.5-7B-Instruct. All three plateaued at 17–19% average—below the Falcon Mamba no-cache baseline. Inspection of predictions shows the LoRA collapses to predicting the most frequent answer per task type (e.g., predicting “garden” for every qa1 example regardless of the actual answer). With 450 training examples and a single-token-greedy evaluation, the LoRA gradient surface favors marginal-distribution shortcuts over context-conditional retrieval. This reinforces a general point about the training regime, not a property of Mamba specifically: low-data SFT

on multi-fact recall tasks tends to learn the per-task answer prior rather than the recall mechanism.

A.4 Learned [CACHE] placement

We tested two alternatives to the heuristic placement rules (loss-selected, uniform-interval) used in the main table: a chain-of-thought SFT adapter that generates traces with [CACHE] markers in-line (Section A.4.1), and a classifier probe that predicts [CACHE] insertion from frozen hidden states (Section A.4.2). Both share an annotation step:

Annotation step (shared). We annotated all 450 qa1–qa3 BABILong samples using Qwen3-235B-A22B-Instruct-2507 (via Tinker). The two variants use different products of this step. For Section A.4.1 we use the full *annotated reasoning traces* the model produces (chain-of-thought text with [CACHE] markers placed mid-reasoning). For Section A.4.2 we use only the *passage-position labels*—the annotator returns the 1-indexed sentence numbers of the minimal supporting facts, yielding 1,067 labels across 448 usable examples (mean 2.37 per example, 2 parse failures).

A.4.1 Chain-of-thought SFT

We treat each (*problem*, *annotated_trace*) pair as a chat example: the user turn is the BABILong question, and the assistant turn is the Qwen3-generated reasoning trace with [CACHE] markers embedded inside the reasoning text. A LoRA adapter is trained with standard prompt-masked cross-entropy. The full pipeline is in Algorithm 1.

Results. $E = 3$ epochs, $LR = 5e-4$. Evaluation used free-generation with `max_new_tokens=512` and regex-based answer extraction (stripping the [CACHE] tokens before matching). Results at 0k filler: qa1=0.960, qa2=0.740, qa3=0.600 (average 0.767). Strong at 0k—higher than every BAM configuration in the main table on these three tasks—but the run was killed after observing qa1/1k=0.36, a sharp drop, indicating the adapter overfit to short-context reasoning. Comparison to the main-table protocol is approximate: free-gen + regex matching is more lenient than the single-token NTM used elsewhere. We do not consider this method a fair entry for Table 2.

A.4.2 Hidden-state probe

A small classifier is trained on top of frozen Falcon Mamba hidden states. The backbone receives a question-then-passage prefix; at every passage

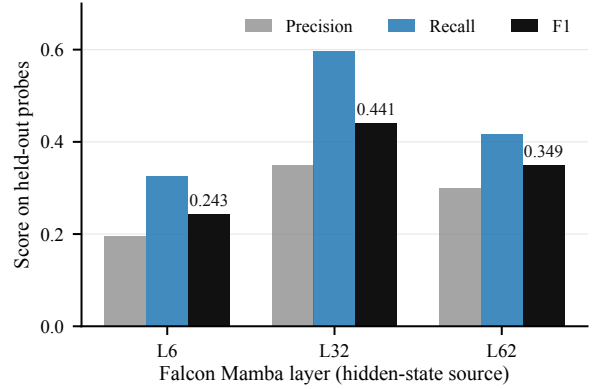


Figure 7: Placement-probe held-out F1 by Falcon Mamba layer of hidden-state extraction. Layer 32 (mid-network) is the sweet spot; layer 6 representations are too raw, layer 62 representations are over-specialized for next-token prediction.

sentence-end token we extract the hidden state from a chosen layer and feed it to a 2-layer MLP that predicts $P([\text{CACHE}])$. No new vocab token, no LoRA on the backbone. Algorithm 2 summarizes.

Layer sweep. We swept the extraction layer $\ell \in \{6, 32, 62\}$ (100 epochs each, MLP-only training is fast on cached features).

Figure 7 shows layer 32 wins clearly: F1 0.441 (precision 0.350, recall 0.597), versus 0.243 at layer 6 and 0.349 at layer 62. Mid-network representations carry the most placement-relevant signal.

Downstream BABILong with probe-driven placement.

We then trained a BAM cache module that takes the layer-32 probe’s predicted positions ($\tau = 0.4$) as its placement source, then ran the same BABILong qa1–qa3 evaluation as the main table. Result: 39.4% average, below both loss-selected (54.4) and uniform-interval (49.9). The probe’s 44% F1 placement quality was not sufficient to drive the cache to match a simple training-free heuristic on this task. The signal exists—layer-32 hidden states do partially encode sentence salience—but the precision is too low to feed a downstream cache without poisoning it with off-target writes.

Algorithm 1 Chain-of-thought SFT for placement

Require: BABILong qa1-qa3 raw samples $\{(q_i, P_i, a_i)\}_{i=1}^N$

Require: Annotator π (Qwen3-235B-A22B-Instruct-2507)

Require: Backbone M (Falcon Mamba 7B), tokenizer T

Phase 1: Generate annotated traces

- 1: **for** $i = 1$ to N **do**
- 2: Prompt π with (q_i, P_i, a_i) to produce a reasoning trace r_i with [CACHE] markers inserted at salient reasoning steps

- 3: **end for**

Phase 2: Build chat-format SFT examples

- 4: Register [CACHE] as a special token in T ; expand M 's embedding and LM head
- 5: **for each** i **do**
- 6: $u_i \leftarrow$ user-turn tokens for " $P_i q_i$ "
- 7: $v_i \leftarrow$ assistant-turn tokens for r_i
- 8: $x_i \leftarrow$ [chat-template wrap of (u_i, v_i)]
- 9: $\ell_i[k] \leftarrow -100$ for $k \in$ prompt span; $\ell_i[k] \leftarrow x_i[k]$ otherwise

- 10: **end for**

Phase 3: SFT with LoRA

- 11: Initialize LoRA adapter on $\{x_{\text{proj}}, \text{in}_{\text{proj}}, \text{dt}_{\text{proj}}\}$ (rank $r=32$, $\alpha=64$); mark `embed_tokens`, `lm_head` trainable so [CACHE]'s vocab row updates
 - 12: **for** epoch = 1 to E **do**
 - 13: **for each** (x_i, ℓ_i) shuffled **do**
 - 14: $\hat{y} \leftarrow M(x_i)$; $\mathcal{L}_i \leftarrow \text{CE}(\hat{y}, \ell_i) \triangleright$ loss only on the assistant span
 - 15: Backprop, AdamW step
 - 16: **end for**
 - 17: **end for**
 - 18: **return** LoRA adapter; at inference, free-generate from the user turn and extract the answer at the end of the trace.
-

Algorithm 2 Hidden-state placement probe

Require: Passage-position labels $\{(q_i, P_i, S_i, C_i)\}_{i=1}^N$ where S_i are passage sentences and $C_i \subseteq \{1, \dots, m_i\}$

Require: Frozen backbone M , tokenizer T , layer index ℓ

Phase 1: Extract features at sentence boundaries

- 1: **for each** i **do**
- 2: $\text{tok}_i \leftarrow T(\text{"Question: } q_i \backslash \text{n Passage: " } \parallel \text{ sentences } S_i)$
- 3: $b_i \leftarrow$ token indices marking the last token of each sentence in tok_i
- 4: $h_i \leftarrow$ forward M over tok_i up to layer ℓ
- 5: $X_i \leftarrow h_i[b_i] \triangleright m_i \times d$ matrix of boundary hidden states
- 6: $Y_i \leftarrow$ one-hot vector with $Y_i[j] = 1$ iff $j \in C_i$
- 7: **end for**
- 8: Concatenate: $X \leftarrow [X_1; \dots; X_N]$, $Y \leftarrow [Y_1; \dots; Y_N]$

Phase 2: Train probe

- 9: Initialize MLP $\phi : \mathbb{R}^d \rightarrow \mathbb{R}$ as $d \rightarrow 512 \rightarrow 1$
- 10: **for** epoch = 1 to E **do**
- 11: **for** minibatch (x, y) from (X, Y) **do**
- 12: $\mathcal{L} \leftarrow \text{BCEWithLogits}(\phi(x), y)$
- 13: Backprop, AdamW step
- 14: **end for**
- 15: **end for**

Phase 3: Inference

- 16: At inference, run Phases 1's forward, score each boundary with $\sigma(\phi(h))$, and emit [CACHE] at every boundary above threshold τ .
 - 17: **return** ϕ, τ
-